Twitter/X 媒体下载器

Java 重写版 类图说明

基于详细设计文档整理

版本：V1.1 日期：2026-10-12

1. 文档说明

本文档对系统核心类的职责、主要属性与方法、以及类之间的关系进行说明，作为详细设计的补充，便于开发时理解对象协作方式。类图以文字 + 关系描述形式呈现，可直接用于绘制 UML 类图或指导编码。

2. 总体类关系概览

系统核心协作关系可概括为：

• CLI 命令类创建并配置 Task，交给 TaskScheduler 执行。

• Task 通过 TaskContext 使用 GraphQLClient、MediaDownloader、DownloadRecordRepository 等。

• GraphQLClient 依赖 AccountPool、RateLimiter、TransactionIdGenerator 完成请求。

• MediaDownloader 依赖 NamingTemplate 生成路径，并将结果写入 Repository。

• 配置由 ConfigLoader 加载为 AppConfig，注入到各组件。

3. 交互层类

3.1 MainCommand / 各子命令

职责：解析命令行参数，调用 AppBootstrap / ComponentFactory 装配依赖（装配细节见时序图 §3），创建对应 Task 并提交 TaskScheduler。

类名	说明
MainCommand	根命令，注册子命令（user / tag / reply / text / profile）
UserCommand	用户媒体下载子命令，参数：users、has-retweet、time-range 等、account-binding（可选，Likes / 亮点 任务必填）

TagCommand	Tag / 高级搜索子命令，参数：tag、filter 等
ReplyCommand	评论区下载子命令，参数：user 或 tweet-url
TextCommand	纯文本获取子命令
ProfileCommand	用户主页信息下载子命令

关系：各 Command 依赖 ConfigLoader、TaskScheduler；创建具体 Task 实例。

4. 配置相关类

4.1 ConfigLoader

职责：读取 YAML 配置文件，合并默认值与环境变量，执行校验，返回 AppConfig。

主要方法：

```
AppConfig load(String path)
void validate(AppConfig config)
```

4.2 AppConfig

职责：强类型配置对象，承载所有运行参数。

主要属性（示例）：

```
String savePath
List<AccountConfig> accounts   // name / note（本人账号标记）/ cookie
ProxyConfig proxy
DownloadConfig download   // maxConcurrent, hasVideo, hasRetweet, timeRange, namingTemplate,
enableIncremental, videoVariant, resumeEnabled ...
RateLimitConfig rateLimit

String dbPath                // SQLite 文件路径

LogConfig log                // level / file / maxHistory

QuotaConfig quota            // perAccountDaily / reserveForRetry / estimateTweetsPerCall

ProxyConfig proxy            // mode: single | pool; list
```

关系：被 ConfigLoader 创建；被 TaskContext、GraphQLClient、MediaDownloader 等读取。

5. 客户端与账号相关类

5.1 GraphQLClient

职责：封装对 X GraphQL 接口的调用，统一处理 Header、认证、Transaction-ID、限流与基础重试。

主要依赖：

```
AccountPool accountPool
RateLimiter rateLimiter
TransactionIdGenerator txIdGenerator
```

```
OkHttpClient httpClient
```

主要方法：

```
<T> T execute(String operationName, String queryId, Map<String,Object> variables, Class<T>
type)
JsonNode executeRaw(String operationName, String queryId, Map<String,Object> variables)
```

关系：被各 Task 通过 TaskContext 使用；依赖 AccountPool、RateLimiter、TransactionIdGenerator。

5.2 TransactionIdGenerator

职责：生成 X-Client-Transaction-ID，实现可替换。

主要方法：

```
String generate(String path, String method)
```

关系：被 GraphQLClient 调用；可有默认实现与备用实现。

5.3 Account

职责：表示一个登录账号及其状态。

主要属性：

```
String name
String cookie / authToken / ct0
AccountStatus status   // AVAILABLE / RATE_LIMITED / INVALID
Instant lastUsedAt / cooldownUntil

long quotaUsedToday / quotaRemaining   // 当日已用调用数与剩余配额
```

5.4 AccountPool

职责：管理多账号，提供获取、释放、标记限流/失效等能力。

主要方法：

```
Account acquire()
Account acquire(String preferredName)   // 指定账号（Likes / 亮点 等必须）
void release(Account account)
void markRateLimited(Account account, Duration cooldown)
void markInvalid(Account account)

void markQuotaExhausted(Account account)
```

关系：持有多个 Account；被 GraphQLClient 使用。

5.5 RateLimiter

职责：控制请求并发与最小间隔，处理 Rate Limit 退避。

主要方法：

```
void acquire() throws InterruptedException
void release()
void onRateLimited(Duration backoff)
```

关系：被 GraphQLClient 在发请求前后调用；可与 AccountPool 协同。

6. 任务体系类

6.1 Task（接口）

```
String name()
void execute(TaskContext context) throws Exception
```

6.2 AbstractTask

职责：提供公共逻辑（进度回调、异常包装、通用过滤等），供具体 Task 继承。

关系：实现 Task；被 UserMediaTask 等继承。

6.3 具体 Task 类

类名	职责与要点
UserMediaTask	按用户拉取媒体时间线；支持排除转推、亮点、喜欢、时间范围；解析 MediaItem 列表并交给 Downloader；可声明 accountBinding（AUTO / 指定账号），Likes 与亮点必须绑定本人账号
TagSearchTask	按 Tag 或高级搜索 filter 拉取结果；可同时保存文本
ReplyTask	按用户或推文链接拉取评论区媒体
TextTask	拉取纯文本推文并落盘
ProfileTask	获取并保存用户头像、banner、简介

关系：均实现/继承 Task；通过 TaskContext 访问 Client、Downloader、Repository。

• accountBinding：由 Command 注入、Task 读取；AUTO 表示可轮换，指定账号表示必须使用该账号。

6.4 TaskContext

职责：向 Task 注入运行时依赖，避免 Task 直接依赖全局单例。

主要属性：

```
AppConfig config
GraphQLClient client
MediaDownloader downloader
DownloadRecordRepository repository
ProgressCallback progressCallback

QuotaLedger quotaLedger    // 每日调用配额账本
```

6.5 TaskScheduler

职责：接收一个或多个 Task，按策略调度执行，汇总结果并输出统计。

主要方法：

```
TaskResult run(Task task)
List<TaskResult> runAll(List<Task> tasks)
```

关系：被 CLI 调用；创建 TaskContext 并执行 Task。

7. 下载与存储相关类

7.1 MediaDownloader

职责：并发下载媒体文件，应用命名模板，更新下载记录，支持重试与进度回调。

主要依赖：

```
NamingTemplate namingTemplate
DownloadRecordRepository repository
ExecutorService executor
DownloadResultQueue resultQueue   // 下载结果投递队列，落库由单写线程串行执行
```

主要方法：

```
DownloadSummary download(List<MediaItem> items, ProgressCallback callback)
```

7.2 NamingTemplate

职责：根据模板字符串与上下文变量渲染文件相对路径。

主要方法：

```
String render(MediaItem item, Map<String,String> extra)
```

支持变量示例：{user} {date} {tweetId} {mediaKey} {type} {index} {ext}；其中 {index} 仅表示本轮运行内序号（重跑会变化），持久稳定命名请使用 {tweetId} / {mediaKey}；渲染结果落盘前经 PathSanitizer 清洗。

7.3 DownloadRecordRepository

职责：封装对 download_record 表的访问，提供增量判断与状态更新。

主要方法：

```
boolean exists(String tweetId, String mediaKey)   // 仅 status='SUCCESS' 视为已下载
void save(DownloadRecord record)   // UPSERT：ON CONFLICT(tweet_id, media_key) DO UPDATE
List<DownloadRecord> listFailed()
void clearByUser(String restId, ClearMode mode)   // mode：仅记录 / 记录+文件 / 含空目录

String getCursor(String restId) / void saveCursor(String restId, String lastMaxTweetId)
```

7.4 Database

职责：管理 SQLite 连接与建表，供 Repository 使用；连接参数 journal_mode = WAL、busy_timeout = 5000、synchronous = NORMAL；写入经单写线程队列串行化。

关系：被 DownloadRecordRepository 依赖；启动时把遗留 task_run.status = RUNNING 收敛

为 ABORTED。

8. 领域模型类

类名	说明
User	用户信息：screenName、restId、name、avatarUrl、bannerUrl、description 等
Tweet	推文：tweetId、text、createdAt、user、mediaItems、isRetweet、metrics 等
MediaItem	单条媒体：url、type（image/video/gif）、tweetId、userScreenName、index 等
DownloadRecord	下载记录：对应数据库表字段，含 status、localPath、downloadedAt 等
TaskResult	任务执行结果：成功数、失败数、跳过数、耗时、失败列表等
DownloadSummary	一次下载批次的汇总信息

关系：Tweet 包含多个 MediaItem；User 与 Tweet 关联；DownloadRecord 由 MediaItem 下载结果生成。

字段补充（P5）：

• ProgressCallback：onProgress(long done, long total, String currentTarget)、onItem(MediaItem item, String status)

• TaskResult：taskName、status（SUCCESS / PARTIAL / FAILED / ABORTED）、successCount、failedCount、skippedCount、elapsedMs、failedItems、resumeCursor

• DownloadSummary：successCount、failedCount、skippedCount、downloadedBytes、failedItems

• MediaItem 增 mediaKey（稳定标识）与 variantBitrate（视频码率变体）

9. 关键关系汇总（便于画图）

依赖 / 使用关系（箭头方向：调用方 → 被用方）：

• MainCommand / UserCommand 等 → ConfigLoader、TaskScheduler

• TaskScheduler → Task、TaskContext

• TaskContext → AppConfig、GraphQLClient、MediaDownloader、DownloadRecordRepository

• UserMediaTask 等 → TaskContext（进而使用 Client / Downloader / Repository）

• GraphQLClient → AccountPool、RateLimiter、TransactionIdGenerator、OkHttpClient

• AccountPool → Account（1 对多）

• QuotaLedger → Account（按账号 × 自然日累计调用次数）

• MediaDownloader → NamingTemplate、DownloadRecordRepository

• DownloadRecordRepository → Database

• Tweet → MediaItem（1 对多）；DownloadRecord 由 MediaItem 衍生

继承关系：

• AbstractTask 实现 Task

• UserMediaTask、TagSearchTask、ReplyTask、TextTask、ProfileTask 继承 AbstractTask

10. 建议类图画图顺序

1. 先画任务体系：Task «interface» ← AbstractTask ← 五个具体 Task，以及 TaskContext、TaskScheduler。

2. 再画服务层：GraphQLClient 及其依赖的 AccountPool、Account、RateLimiter、TransactionIdGenerator。

3. 然后画下载与存储：MediaDownloader、NamingTemplate、DownloadRecordRepository、Database。

4. 补充领域模型：User、Tweet、MediaItem、DownloadRecord、TaskResult。

5. 最后用依赖箭头把 CLI → Scheduler → Task → Context → Client/Downloader/Repository 串起来。

11. 补充说明

以上类名与方法为设计阶段约定，实现时可按实际需要微调，但建议保持模块边界与核心职责不变。绘制 UML 时，可将“依赖”表示为虚线箭头，继承为实线空心三角，接口实现为虚线空心三角。若需要 PlantUML / Mermaid 源码版本，可在此说明基础上再生成一版文本图。

12. 文档效力与修订记录

文档效力顺序：本套文档共五份（需求分析、详细设计、类图说明、时序图与关键流程图、数据库结构说明）。效力顺序为：需求分析 > 详细设计 > 类图说明 / 时序图与关键流程图 / 数据库结构说明（三者同级，冲突时以详细设计为准）。出现冲突时，以顺序在前的文档为准。

修订记录：

版本	日期	说明	备注
V1.1	2026-10-12	依据评审意见整改：H1/H5 Repository.exists 改按 (tweetId, mediaKey) 且仅认 SUCCESS（§7.3）；H2 Account/AccountPool 增配额字段与 markQuotaExhausted，AppConfig 增 quota / dbPath / log（§4.2、§5.3、§5.4）；H3 MediaDownloader 增结果投递队列（§7.1）；H6 增 accountBinding（§3.1、§6.3、§6.4）；S2 配置项补全（§4.2）；	评审后首版

		S6 MediaItem 增 mediaKey / variantBitrate（§8）；S9 命名模板稳定键（§7.2）；S14 增文档效力顺序与本表；P1 统一 operationName（§5.1）；P5 补三个模型的字段（§8）；P6 装配下沉 AppBootstrap（§3.1）；S8 clearByUser 三档（§7.3）	

— 文档结束 —